

PLAN.MD — Déroulé de la démo EnerFlex (≈ 50 min, ou 25 min en version courte)

*Le message de la démo : sur une application existante, une équipe d'agents IA livre une user story de bout en bout — mais **l'humain relit et valide trois fois** : le plan, les tests, le code. L'IA va vite ; la relecture fait la qualité. Tout se termine par un rapport de livraison et une spécification technique à jour, sans qu'on les ait écrits à la main.*

0. Avant d'entrer en salle (10 min, la veille)

```
cd C:\dev\workspaces\ai-demo-2026\demo3
npm install
npm test                # tout vert, couverture > 70 % des deux côtés
npm run dev              # API :3001 + UI :5173
```

Ouvrir <http://localhost:5173> et vérifier :

- le tableau de bord d'**août 2026** (mois par défaut), avec ses alertes ;
- la fiche **LIL-02 — Entrepôt de Lesquin** : la jauge de puissance reste très basse (site surdimensionné) ;
- la fiche **LYO-01 — Usine de Vénissieux** : dépassements, alerte Critique, ligne de pénalité non nulle (site sous-dimensionné).

Ce sont les deux sites qu'on regardera après la story : l'un doit recommander de **réduire**, l'autre d'**augmenter** la puissance souscrite.

Puis, dans Claude Code (ouvert dans `demo3/`) :

- vérifier que les agents apparaissent : `/agents` → 7 agents ;
- vérifier que les skills apparaissent : taper `/` → `implement-us`, `story-readiness`, ... ;
- vérifier que le serveur MCP Chrome DevTools est connecté (`/mcp`) — c'est l'agent `reviewer` qui s'en sert pour voir l'écran (lui seul a ces outils dans sa fiche) ;
- `git status` propre, `git log -1` noté : c'est le point de retour si tout doit être rejoué (`git reset --hard <commit>`).

Prévoir deux fenêtres côte à côte : Claude Code à gauche, le navigateur (l'app, puis les documents Markdown) à droite.

Durées réelles, mesurées sur une exécution complète du 8 septembre 2026 (temps machine seul, hors ce qu'on raconte) : BA 3 min, architecte 7 min, `test-dev` 9 min, les deux devs en parallèle 5 min, `reviewer` 11 min, `tech-writer` 7 min — **environ 45 min d'agents**, donc 50 à 60 min avec les commentaires et les relectures. Si le créneau est d'une demi-heure, jouer la **version courte** (§6) plutôt que d'accélérer : les portes humaines sont le sujet de la démo, elles ne se sautent pas.

1. Poser le décor (3 min)

À l'écran : l'application. À dire, en trois phrases :

- 1. « Voici EnerFlex : un client industriel, 12 sites, sa consommation, sa facture calculée depuis la grille du contrat — heures pleines / heures creuses, abonnement à la puissance souscrite, pénalités de dépassement — et ses alertes. » (montrer LYO-01 : la pénalité.)
- 2. « L'application existe déjà, avec ses tests, sa spec technique et ses règles de développement. » (montrer docs/SPEC-TECHNIQUE.md §4.3, la formule de la facture.)
- 3. « On va lui ajouter une fonctionnalité que le client vient de demander. » (ouvrir specs/US-006-optimisation-puissance-souscrite.md , lire le contexte métier et l'exemple.)

Puis montrer .claude/agents/ : sept fichiers Markdown, « ce sont les fiches de poste de l'équipe ». Ouvrir business-analyst.md dix secondes : un rôle, un périmètre, des interdictions.

2. Lancer la story (le prompt unique)

Dans Claude Code :

```
Utilise le skill implement-us sur US-006.
```

Le skill enchaîne les agents et **s'arrête à chaque porte humaine**. Le déroulé ci-dessous dit, pour chaque étape, ce qui se passe, ce qu'on montre, et ce qu'on répond.

Astuce de rythme : lancer ce prompt **avant** de poser le décor (§1), pas après. Le BA travaille pendant les trois minutes de présentation ; sa revue est prête quand on arrive dessus, et on économise le premier temps mort.

Variante étape par étape (si l'on préfère lancer chaque agent soi-même) : les prompts sont en annexe A.

3. Étape par étape

3.1 — L'agent BA relit la story (≈ 3 min)

Ce qui se passe : business-analyst lit la story, la spec technique et le code, et rend docs/revues/US-006-revue.md avec un verdict et des questions.

Ce qu'on montre : le rapport, côté navigateur. Les six axes, puis les constats. Lire les questions **à voix haute** : c'est le moment où le public comprend que l'agent *lit* au lieu d'obéir.

Les manques laissés volontairement dans la story (l'agent devrait en trouver au moins deux ; s'il en trouve d'autres, tant mieux — répondre de la même façon) :

Manque	Question probable	Réponse à donner
Le pas des puissances candidates n'est pas dit (l'exemple suggère 10 kVA sans l'énoncer)	« Quelles puissances sont candidates ? »	Multiples de 10 kVA , de 10 kVA jusqu'à la puissance maximale relevée du mois arrondie à la dizaine supérieure.
Égalité de coût entre deux puissances non tranchée	« Laquelle retenir à coût égal ? »	La plus élevée (prudence : moins de dépassements).
Cas « déjà optimale » absent (CA-5 ne prévoit que réduire / augmenter)	« Que montrer si la puissance actuelle est déjà la meilleure ? »	Économie 0,00 € , sens « Conserver la puissance souscrite » — on accepte d'ajouter ce troisième libellé à CA-5.
Arrondi des coûts intermédiaires	« Arrondir chaque coût ou seulement l'économie ? »	Calcul sans arrondi , arrondi au centime à l'affichage de chaque montant.

Ce qu'on répond : une ligne par question, dans la conversation. Dire au public : « Ces réponses font partie du besoin ; elles seront transmises à l'architecte et reportées dans la story par un humain. »

Attention, deux surprises vérifiées en répétition :

- Le verdict tombe à « **Non prête** », pas « À clarifier » — le skill prévoit alors « on s'arrête, l'humain réécrit la story ». **Ne pas s'arrêter** : dire au public que répondre aux questions *est* la réécriture, et enchaîner. C'est même le meilleur moment de l'étape.
- Les questions ne sont pas formulées comme le tableau ci-dessus : le BA demande plutôt « pas de 1 kVA ? » et « à égalité, la plus basse ? ». Les réponses préparées restent valables, mais il faut les **reformuler en direct** (« pas de 10, et à égalité la plus haute »), pas les lire.
- Le BA sort de lui-même le point le plus fort de la démo : avec cette grille, l'optimum est un **plateau** (2,85 € gagnés par kVA retiré contre 0,95 € par kWh de dépassement → toute puissance avec exactement trois heures de dépassement coûte pareil), donc l'exemple de la story n'a une réponse unique que si le pas vaut 10 kVA. Le garder pour la fin serait dommage : le commenter ici.

3.2 — L'agent architecte écrit le plan → II porte humaine n° 1 (≈ 7 min)

Ce qui se passe : `architect` produit `docs/architecture/US-006-plan.md` : la règle comme fonction pure (`recommanderPuissance`), l'exemple chiffré recalculé, la route (`GET /api/sites/:id/optimisation?mois=`), le type de réponse, le bloc d'écran, la **liste nominative des tests**, deux alternatives, un ADR si besoin.

Pendant qu'il travaille — c'est le temps mort le plus long avant les devs : montrer `.claude/rules/` (« un agent, un dossier ; les tests ne sont jamais affaiblis ; la règle vit dans `domain/` ») et la fiche `architect.md`, notamment son budget de 150 lignes.

Ce qu'on montre : le plan dans le navigateur. Insister sur :

- §2 : la règle tient en une signature et un exemple — « c'est ça qu'on relit, pas du code » ;
- §5 : le plan de tests — « chaque critère a déjà son test, nommé en français » ;
- §6 : les alternatives — « l'agent ne présente jamais une seule option ».

Ce qu'on fait : relire vraiment, puis répondre « Le plan est validé. » Astuce : demander **une** modification pour montrer la boucle — par exemple « ajoute un test pour un mois sans aucun relevé au-dessus de 10 kVA » — puis valider.

3.3 — L'agent test-dev écrit les tests, rouges → II porte humaine n° 2 (≈ 9 min)

Ce qui se passe : `test-dev` écrit `backend/tests/optimisation.test.ts`, complète `api.test.ts`, ajoute les tests d'UI ; il lance les suites : les nouveaux tests **échouent** (module absent), les anciens passent.

Ce qu'on montre : un fichier de test. Les noms en français se lisent comme la story ; les valeurs attendues sont **calculées à la main en commentaire** (« $P = 90 : 256,50 + 71,25 = 327,75$ »). Puis le terminal — **annoncer les chiffres avant de les afficher**, ils sont plus spectaculaires que prévu : côté backend « 6 failed, 69 passed » **plus une suite entière qui ne se collecte pas** (`Cannot find module '../src/domain/optimisation.js'` : le module n'existe pas encore, c'est le rouge le plus pur qui soit) ; côté frontend « 10 failed, 50 passed », dont **quatre tests préexistants** de `SiteDetailPage` — ils cassent parce que `getOptimisation` n'est pas encore exporté, le plan l'avait anticipé, et ils redeviendront verts tout seuls à l'étape suivante. Le dire **avant** que quelqu'un dans la salle croie à une régression.

Ce qu'on fait : relire un ou deux tests, puis « Les tests sont validés. »

3.4 — Les agents dev implémentent, en parallèle (≈ 5 min)

Ce qui se passe : `node-dev` (domaine → service → route) et `react-dev` (type → api → hook → composant → page) tournent **en même temps**, chacun dans son dossier, jusqu'au vert. `react-dev` vérifie son écran par les tests Testing Library et par un appel `curl` au proxy Vite ; c'est le `reviewer`, à l'étape suivante, qui ouvre vraiment Chrome (lui seul a les outils MCP dans sa fiche).

Ce qu'on montre : c'est l'étape la plus courte et la plus impressionnante — deux agents qui écrivent en parallèle sans se marcher dessus. À la fin : `npm test` vert des deux côtés, couverture affichée (backend 88 tests, frontend 60).

3.5 — L'agent reviewer relit → II porte humaine n° 3 (≈ 11 min)

Ce qui se passe : `reviewer` (lecture seule) rend `docs/revues/US-006-revue-code.md` : critères couverts, fidélité au plan, tests non affaiblis, règles, écran vu dans Chrome, constats par sévérité.

Ce qu'on montre : le rapport ; puis `git diff --stat` ; puis **l'application** : `/sites/LIL-02` → « Réduire la puissance souscrite » (630 → 250 kVA, ≈ 1 083 € HT d'économie par mois) ; `/sites/LY0-01` → « Augmenter la puissance souscrite » (1 250 → 1 480 kVA, ≈ 5 038 € HT par mois : les pénalités disparaissent presque).

Rappel du point de plateau déjà sorti par le BA au §3.1 : l'optimum tolère environ trois heures de dépassement par mois, donc tous les sites reçoivent une recommandation légèrement sous leur pointe. C'est le contrat qui le veut, pas l'algorithme.

Ce que le reviewer trouve, en répétition : zéro `BLOQUANT`, un `À CORRIGER` (le test qui vérifie les chemins d'URL du module `api/` n'a pas été complété : une faute de frappe dans l'URL passerait les 60 tests) et quelques suggestions. C'est **la bonne nouvelle de la démo** : le constat est réel, précis, et il porte sur un trou de couverture qu'aucune suite verte ne montrait. Le corriger devant le public (une ligne, par `test-dev`, pas par un dev) puis relancer la suite prend une minute et vaut tous les discours sur la relecture.

Ce qu'on fait : relire un fichier de code (le domaine, `optimisation.ts` : une vingtaine de lignes), puis « Le code est validé. »

3.6 — L'agent tech-writer clôture (≈ 7 min)

Ce qui se passe : `tech-writer` rend `docs/rapports/US-006-rapport.md` (+ PDF) et met à jour `docs/SPEC-TECHNIQUE.md` (§4 nouvelle règle, §5 nouvelle route, §6 nouveau bloc, §9 historique, version 1.1). Puis commit — **à ne faire que si l'on garde le résultat** ; en répétition, on rollback plutôt (§5).

Ce qu'on montre : le PDF du rapport (le tableau du parcours, avec les trois portes humaines datées) et le diff de la spec technique.

4. Conclure (2 min)

- Sept agents, trois relectures humaines, une story livrée avec ses tests, sa revue, son rapport et sa spec — **en 30 minutes**, sur une base existante.
- Ce qui a fait la qualité : la revue du besoin avant le code, les tests avant le code, un relecteur qui ne corrige pas, et un humain qui dit « oui » trois fois.
- Tout est dans le dépôt : les fiches de poste (`.claude/agents/`), les pratiques (`.claude/skills/`), les règles (`.claude/rules/`). « Ce n'est pas de la magie : c'est de l'organisation, écrite en Markdown. »

5. Si quelque chose tourne mal

Problème	Parade
Le BA rend « Prête » sans question	Dire que la story était bien écrite, et provoquer la boucle à la porte n° 1 (demander une modification du plan).
Le BA rend « Non prête » (c'est le cas observé)	Ne pas s'arrêter malgré la consigne du skill : répondre aux questions, dire que c'est cela, réécrire la story, et enchaîner sur l'architecte.
Un agent dépasse son dossier	Le dire au public : « c'est exactement ce que le reviewer va relever » — et laisser le reviewer le trouver.
Les devs dépassent 8 min	Continuer à parler des règles et des skills ; le résultat arrive.
Vite affiche un autre port que 5173	Un autre projet occupe 5173 : utiliser le port affiché dans le terminal (5174 ...), le proxy /api fonctionne quel que soit le port.
Un port est occupé	<code>Get-NetTCPConnection -LocalPort 3001,5173 Select OwningProcess</code> puis <code>Stop-Process -Id <pid></code> — à taper soi-même dans le terminal : lancée par un agent, la commande échoue en « accès refusé ».
Tout rejouer	<code>git reset --hard <commit noté en §0></code> puis <code>git clean -fd backend frontend docs</code> , puis relancer <code>npm run dev</code> . Le <code>reset</code> seul laisse les fichiers créés (domaine, composant, revues, rapport).
Plus que 15 minutes	Jouer les étapes 3.1 à 3.3 seulement (revue, plan, tests rouges) : c'est déjà le cœur du message — relire avant de coder. Voir la version courte, §6.
Un agent écrit dans <code>.claude/</code> ou dans un dossier de process	C'est un réglage humain, pas la story : le reviewer le signalera comme hors périmètre. Le lui accorder à voix haute — c'est la preuve que la garde fonctionne.

6. Version courte (≈ 25 min)

Quand le créneau est d'une demi-heure, ne pas accélérer le déroulé complet : jouer **3.1 → 3.3 seulement**, puis conclure.

Temps	Étape
0 min	Lancer <code>implement-us</code> (§2), puis poser le décor pendant que le BA travaille (§1).
3 min	La revue du BA, ses questions, les réponses (§3.1).
5 min	L'architecte travaille : montrer <code>.claude/rules/</code> et les fiches d'agents.
12 min	Le plan, la porte humaine n° 1 (§3.2).
15 min	<code>test-dev</code> travaille : montrer la story et ses critères en regard du plan de tests.
22 min	Les tests rouges, la porte humaine n° 2 (§3.3).
24 min	Conclure (§4) en montrant qu'il ne reste plus qu'à laisser les devs passer au vert.

Le message tient entier dans ces trois étapes : **on relit le besoin, puis le plan, puis les tests — avant qu'une ligne de code soit écrite.**

Annexe A — Prompts étape par étape

1. Revue du besoin

Utilise le skill story-readiness sur US-006 : délègue à l'agent business-analyst.

2. Plan

Utilise le skill architecture-plan pour US-006 : délègue à l'agent architect avec docs/revues/US-006-revue.md et mes réponses : (1) multiples de 10 kVA jusqu'à la puissance max relevée arrondie à la dizaine supérieure ; (2) à coût égal, la plus élevée ; (3) si déjà optimale : économie 0,00 € et « Conserver la puissance souscrite » ; (4) calcul sans arrondi, arrondi au centime à l'affichage ; (5) les deux montants sont des coûts partiels (abonnement + pénalités), à libeller pour ne pas les confondre avec le « Montant HT » de la facture.

Contexte à lire, rien de plus : docs/SPEC-TECHNIQUE.md §4.3 et §5, backend/src/domain/tarification.ts, backend/src/routes/sites.ts, frontend/src/pages/SiteDetailPage.tsx.

Plan en 150 lignes maximum.

3. Tests

Lance l'agent test-dev sur US-006 avec docs/architecture/US-006-plan.md : uniquement des fichiers de test, suite rouge pour la bonne raison. Puis liste-moi les tests écrits.

4. Implémentation

Lance en parallèle node-dev (backend/**) et react-dev (frontend/**) sur US-006 avec le plan et les tests rouges. Ils ne modifient pas les tests. Puis npm run typecheck && npm run lint && npm test.

5. Revue

Lance l'agent reviewer sur US-006 avec le plan et le commit de départ <sha>. L'app tourne sur :5173, qu'il vérifie l'écran dans Chrome (LIL-02 réduire, LYO-01 augmenter).

6. Clôture

Lance l'agent tech-writer sur US-006 : rapport de livraison (md + PDF) et mise à jour de docs/SPEC-TECHNIQUE.md. Puis, si l'on garde le résultat, commit « feat(enerflex): US-006 recommandation de puissance souscrite ».

Annexe B — Pourquoi cette story

- **Reconnaissable en cinq secondes** : tout le monde dans la salle sait ce qu'est une puissance souscrite et une pénalité de dépassement.
- **Une vraie règle métier** (un petit algorithme d'optimisation), donc de vrais tests de domaine avec des valeurs calculées à la main — le TDD a quelque chose à mordre.
- **Une tranche complète mais courte** : une fonction de domaine, une route, un bloc d'écran. Pas de nouvelle entité, pas de migration, pas de formulaire.
- **Deux résultats opposés visibles à l'écran** (LIL-02 réduire, LYO-01 augmenter) : la démo finit sur l'application, pas sur un terminal.
- **Des manques volontaires** dans la story, pour que le BA ait quelque chose à dire.